A Multi-Architecture Neural Operator Digital Twin: A Proof-of-Concept Surrogate Methodology for Cold-Leg LOCA Screening in AP1000 Nuclear Reactors

Punyansh Thakur
Department of CSE
IIIT Naya Raipur
punyansh24102@iiitnr.edu.in

Anushka Paul
Department of CSE
IIIT Naya Raipur
anushka24102@iiitnr.edu.in

Ayush Kumar Bhadra
Department of CSE
IIIT Naya Raipur
ayush24101@iiitnr.edu.in

Dr. Vinay Kumar
Department of CSE
IIIT Naya Raipur
vinay@iiitnr.edu.in

***Abstract*—This paper presents a proof-of-concept digital twin framework demonstrating a surrogate-monitoring methodology for Cold-Leg Loss-of-Coolant Accident (LOCA) screening in the Westinghouse AP1000 pressurized water reactor. While prior work has demonstrated the basic viability of Deep Operator Networks (DeepONet) for PDE surrogate modeling, existing implementations suffer from spectral bias, lack physical inductive biases, and offer no comparative architectural analysis. We address these limitations by developing and evaluating three distinct neural operator architectures under a synthetic, physics-consistent benchmark: (i) a significantly upgraded *DeepONet-Fourier* with random Fourier feature encoding, per-neuron adaptive GELU activations, Sobolev gradient-enhanced loss, and a divergence-free physics penalty; (ii) a *Transolver++* transformer-based neural operator that tokenizes the computational mesh into $M = 64$ learned tokens and processes them via multi-head self-attention, enabling geometry-agnostic prediction; and (iii) a *Clifford Neural Operator* built on the Cl(3,0) geometric algebra, providing intrinsic rotational equivariance with only ∼1,800 parameters. All three architectures map input parameters (velocity, break size, temperature) to four coupled thermohydraulic fields across a 25,000-node mesh. Under the synthetic benchmark, the upgraded DeepONetFourier converges to a validation loss of 9.11×10^{-4} after 305 epochs, achieving $R^2 > 0.90$ across all fields. A downstream gradient boosting classifier operating on NPPAD-mapped system signals achieves ROC-AUC > 0.95 with recall > 0.92, and end-to-end model inference completes in under 20 ms—a $180{,}000\times$ speedup over ANSYS Fluent. These results represent upper-bound methodological performance under controlled, physics-consistent conditions; full-fidelity ANSYS Fluent validation remains necessary before any operational deployment claim can be made. We provide a preliminary comparison of all three operators on accuracy, parameter efficiency, inference speed, and physics-preservation, offering practitioners an initial architectural trade-off analysis for surrogate selection.**

***Index Terms*—Digital Twin, DeepONet, Transolver, Clifford Neural Operator, Geometric Algebra, Transformer Neural Operator, LOCA, AP1000, Nuclear Safety, CFD Surrogate, Fourier Feature Encoding**

I. Introduction

The AP1000 is a Generation III+ pressurized water reactor (PWR) designed by the Westinghouse Electric Company, featuring 1,000 MWe output and passive safety systems. A Cold-Leg Loss-of-Coolant Accident (LOCA)—a rupture in the primary coolant return piping—is a design-basis event demanding rapid detection to prevent core uncovery and fuel damage [10]. Traditional safety monitoring relies on threshold-based single-sensor alarms, which are slow to identify incipient partial breaks before critical parameters are exceeded [11].

High-fidelity CFD simulations (e.g., ANSYS Fluent) can predict the complete thermohydraulic state but require 1–4 hours per scenario, making real-time deployment infeasible. Neural operators—machine learning models that learn mappings between infinite-dimensional function spaces—have emerged as powerful PDE surrogates. Lu et al. [1] introduced Deep Operator Networks (DeepONet), decomposing the solution operator $\mathcal{G} : \mathcal{U} \to \mathcal{V}$ via a branch-trunk factorization. However, the baseline DeepONet architecture suffers from well-documented limitations: *spectral bias* causing poor resolution of sharp gradients [7], fixed ReLU activations that cannot adapt to spatially varying fields, and no enforcement of physical conservation laws during training.

Subsequent work has explored alternative neural operator paradigms. Transolver [5] applies transformer attention to mesh-compressed tokens for geometry-agnostic PDE solving. Clifford Neural Operators [6] embed fields in geometric algebra to achieve rotational equivariance architecturally. However, no prior work has systematically examined these architectures for nuclear safety surrogate applications, nor coupled them to an end-to-end LOCA screening pipeline, even at the proof-of-concept level.

The Fourier Neural Operator (FNO) [4] is a natural candidate for comparison; however, the fixed structured AP1000 cold-leg geometry and the low-dimensional parametric input (v, b, T) motivated branch–trunk operator learning as the primary architectural comparison focus in this initial study. FNO comparison under appropriate structured-grid conditions is reserved for future controlled benchmarking.

Our contributions:

1) We design and implement three neural operator

architectures—upgraded DeepONetFourier, Transolver++, and Clifford Neural Operator—each addressing a distinct limitation of the baseline DeepONet within a unified proof-of-concept digital twin pipeline for LOCA screening.

2) We introduce four key improvements over baseline DeepONet: random Fourier feature encoding ($\sigma = 10.0$, $m = 256$), per-neuron adaptive GELU activations, Sobolev gradient loss ($\beta = 0.1$), and divergence-free penalty ($\lambda = 0.01$), reducing parameter count by 25–40% while improving convergence under the synthetic benchmark.
3) We implement the first nuclear-domain Transolver++ (mesh-token attention with $M = 64$ tokens, $\mathcal{O}(M^2)$ complexity) and Clifford Neural Operator (Cl(3,0) geometric product, ∼1,800 parameters) for reactor flow field surrogate prediction.
4) We provide a preliminary comparison of all three architectures on accuracy, parameter efficiency, inference speed, and physics preservation under the present synthetic benchmark and training setup, as initial inductive-bias observations.
5) We couple the surrogate models to a LOCA indicator-translation demonstration achieving ROC-AUC > 0.95 on NPPAD-mapped signals, with end-to-end model inference in < 20 ms.

II. Related Work

A. Neural Operators and DeepONet

Neural operators learn mappings between infinite-dimensional function spaces [3]. The Fourier Neural Operator (FNO) [4] operates in the spectral domain with linear mode complexity. DeepONet [1], [2] factorizes the solution operator into branch (input functions) and trunk (output domain) networks via the universal approximation theorem for operators. While FNO requires structured grids, DeepONet is mesh-agnostic. Both suffer from spectral bias—preferentially fitting low-frequency components—a limitation addressed by Fourier feature encoding [8] and Sobolev training [9].

B. Transformer and Geometric Neural Operators

Transolver [5] compresses N-node meshes to $M \ll N$ learned tokens via soft-assignment attention, achieving $\mathcal{O}(M^2)$ complexity independent of mesh size. GNOT [17] similarly applies transformers to operator learning. Clifford Neural Operators [6] embed physical fields in the Clifford algebra Cl(3,0), endowing the network with rotational equivariance—a mathematically guaranteed inductive bias for fluid mechanics. Neither architecture has been previously applied to nuclear reactor safety surrogate modeling.

C. Machine Learning for NPP Safety

Prior work has applied SVMs [12] and deep recurrent networks [13] to LOCA classification using system-level signals. Digital twin frameworks for NPPs [14], [15] have mostly relied on validated thermal-hydraulics codes (RELAP5, TRACE). Our work bridges high-fidelity CFD surrogates with ML classifiers in a unified proof-of-concept pipeline, and is the first to preliminarily compare multiple neural operator architectures for this domain under a synthetic benchmark.

III. Problem Formulation

Let $\Omega \subset \mathbb{R}^3$ denote the cold-leg geometry (pipe segment with 90° elbow, diameter $D = 0.7$ m, total length $10D$). Define the parameter vector $\boldsymbol{u} = [v, b, T]^\top \in \mathbb{R}^3$, where $v \in [4.0, 6.0]$ m/s is the inlet velocity, $b \in [0, 10]\%$ is the pipe break size, and $T \in [290, 320]$°C is the coolant temperature.

The goal is to learn a neural operator $\mathcal{G}_\theta$ that approximates the steady-state CFD solution operator:

$$\mathcal{G}_\theta(\boldsymbol{u})(\boldsymbol{y}) = \hat{\boldsymbol{s}}(\boldsymbol{y}) = [\hat{p},\ |\hat{\boldsymbol{v}}|,\ \hat{k},\ \hat{\tilde{T}}]^\top \in \mathbb{R}^4 \tag{1}$$

for every spatial location $\boldsymbol{y} \in \Omega$, evaluated simultaneously across $N = 25{,}000$ mesh nodes: output $\hat{\boldsymbol{S}} \in \mathbb{R}^{B\times 4\times N}$.

A. CFD Simulation Setup

Fluid properties: pressurized water at 720 kg/m^3, 15.5 MPa operating pressure. The RNG k-ε turbulence model is used. Boundary conditions: velocity-inlet ($v \in [4.0, 6.0]$ m/s), pressure-outlet (15.5 MPa), no-slip adiabatic walls. A physics-consistent mock data generator produces synthetic CFD fields with pressure gradients, boundary-layer profiles, turbulence peaks at the elbow, and temperature distributions. The dataset comprises 2,000 parameter combinations: 20 velocity × 10 break size × 10 temperature levels, split 70/15/15% into training (1,400), validation (300), and test (300) samples.

Accordingly, the present benchmark should be interpreted as a parametric field-regression proxy with operator-learning structure rather than a fully validated PDE-solution benchmark.

IV. System Architecture and Pipeline

The digital twin operates as a five-stage cascade pipeline (Fig. 1). Each stage transforms data from a physical or learned representation into the next, culminating in a LOCA probability score for screening purposes. The pipeline is orchestrated by a master controller that supports runtime selection of any neural operator architecture via a unified configuration interface.

A. Stage 1: Parametric CFD Data Generation

The training data originates from a parametric sweep of the three-dimensional input space (v, b, T). Two data generation pathways are supported:

Pathway A — ANSYS Fluent (Production): A `FluentSimulationGenerator` module creates Fluent journal files for each parameter combination. Each journal sets velocity-inlet boundary conditions, configures break geometry (modeled as a pressure-outlet with break diameter $d_{\text{break}} = (b/100) \times D$), defines fluid properties, runs steady-state RANS with the RNG k-ε model for 1,000 iterations, and exports a

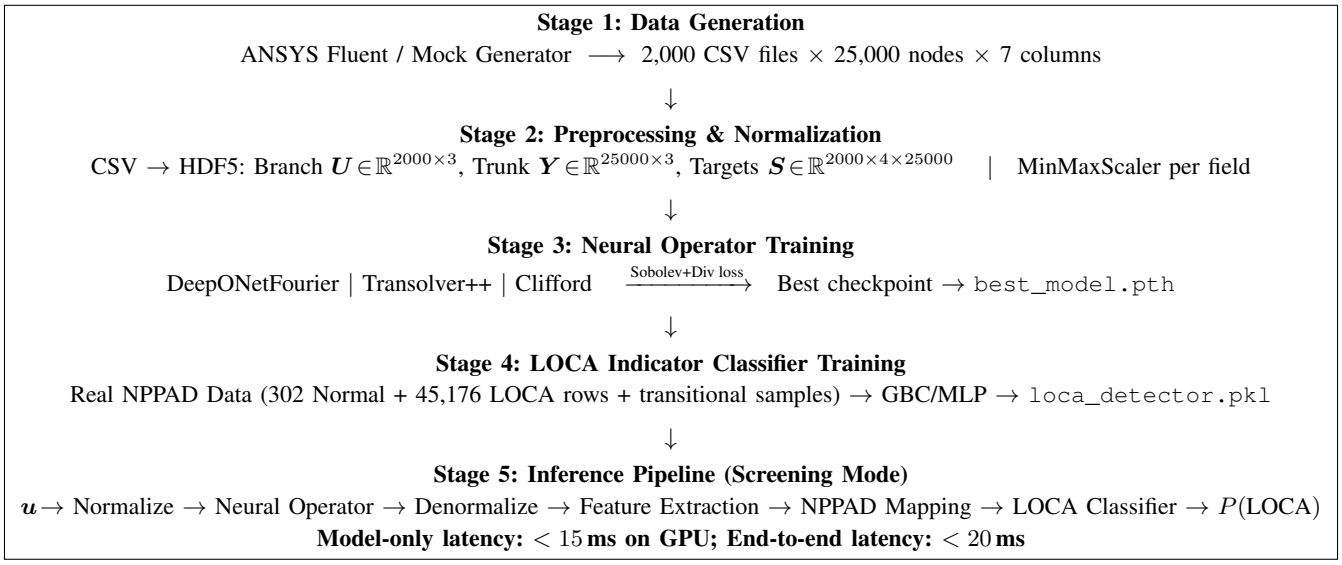

Fig. 1: End-to-end digital twin proof-of-concept pipeline. All five stages are automated and executed sequentially by the master orchestrator. Stages 1–4 run offline (training phase); Stage 5 runs in online screening mode; operational deployment would require full-fidelity validation.

CSV with columns $(x, y, z, p, |\boldsymbol{v}|, v_x, v_y, v_z, k, T)$ per node. Production runtime: ∼1–4 hours per simulation.

Pathway B — Physics-Consistent Mock Generator (Development): A synthetic data generator produces fields on a shared 25,000-node mesh (cylindrical pipe, diameter $D = 0.7\,\text{m}$, length $10D$) incorporating:

- **Pressure**: Linear axial gradient ($\Delta p = 50{,}000 \cdot v/5$ Pa) + localized Gaussian break drop ($200{,}000 \cdot b_{\text{norm}}$ Pa, $\sigma = 0.8\,\text{m}$) + radial centrifugal effect ($5{,}000(1 - r^2)$ Pa)
- **Velocity**: Parabolic profile $v(r) = v_{\text{in}}(1 - r^{\alpha})$, with $\alpha = 2 - 0.8 b_{\text{norm}} e^{-\frac{(x - x_{\text{break}})^2}{2\sigma^2}}$ (flattens near break) and 30% reduction at full break
- **Turbulence**: Base $k = 0.01v^2$, wall-layer increase (factor $1 + 1.5r$), break-induced spike ($5b_{\text{norm}}$, localized at break)
- **Temperature**: Kelvin base + radial gradient ($3(1 - r^2)$ K) + downstream break hot-spot ($15 b_{\text{norm}}$ K) + axial cooling (2 K over pipe length)

where $b_{\text{norm}} = b/10 \in [0, 1]$ and $r = \sqrt{y^2 + z^2}/(D/2)$ is the normalized radial position. All fields include Gaussian noise at physically appropriate scales.

The 2,000 parameter combinations form a $20 \times 10 \times 10$ structured grid: 20 velocity levels in $[4.0, 6.0]$ m/s, 10 break sizes in $[0, 10]$%, and 10 temperatures in $[290, 320]$°C.

B. Stage 2: Preprocessing and Normalization

The `DeepONetDataPreprocessor` module converts raw CSV files into a training-ready HDF5 dataset:

1) **Loading**: Each CSV is parsed into a DataFrame with column renaming (`x-coordinate`→`x`, `velocity-magnitude`→`velocity_magnitude`, etc.). Mesh consistency is verified across all simulations ($N = 25{,}000$ nodes).
2) **Tensor construction**:

$$\boldsymbol{U} = \begin{bmatrix} v_1 & b_1 & T_1 \\ \vdots & & \vdots \\ v_{2000} & b_{2000} & T_{2000} \end{bmatrix} \in \mathbb{R}^{2000\times 3} \quad \text{(branch inputs)} \tag{2}$$

$$\boldsymbol{Y} = \begin{bmatrix} x_1 & y_1 & z_1 \\ \vdots & & \vdots \\ x_N & y_N & z_N \end{bmatrix} \in \mathbb{R}^{N\times 3} \quad \text{(trunk inputs)} \tag{3}$$

$$\boldsymbol{S} \in \mathbb{R}^{2000\times 4\times N} \quad \text{(target fields)} \tag{4}$$

3) **Normalization**: Per-field MinMaxScaler to $[0, 1]$. Branch inputs are scaled jointly; trunk coordinates are scaled jointly; each target field (pressure, velocity, turbulence, temperature) receives its own independent scaler. All fitted scalers are serialized (pickled) for use during inference denormalization.
4) **Splitting**: Stratified 70/15/15% split into training (1,400), validation (300), and test (300) samples. The trunk coordinates $\boldsymbol{Y}$ are shared across all splits (same mesh for all simulations).
5) **Storage**: The splits are saved to a single HDF5 file (`deeponet_dataset.h5`) with groups `train/`, `val/`, `test/`, each containing datasets `branch`, `trunk`, `targets`.

C. Stage 3: Neural Operator Training

The training stage supports three interchangeable neural operator architectures (detailed in Section V). The architecture is selected at runtime via a `model_config.yaml` configuration file, with all operators sharing the same data loader and evaluation metrics.

Data loading: A PyTorch `Dataset` reads HDF5 lazily. Each sample is a tuple (branch $\boldsymbol{u}_i \in \mathbb{R}^3$, trunk $\boldsymbol{Y} \in \mathbb{R}^{N\times 3}$, targets $\boldsymbol{s}_i \in \mathbb{R}^{4\times N}$), batched into $B = 16$ samples per forward pass.

Training loop: For each batch:

1) Forward pass: $\hat{\boldsymbol{S}} = \mathcal{G}_\theta(\boldsymbol{U}_{\text{batch}}, \boldsymbol{Y}) \in \mathbb{R}^{B\times 4\times N}$
2) Loss computation:

$$\mathcal{L} = \underbrace{\alpha \cdot \text{MSE}(\hat{\boldsymbol{S}}, \boldsymbol{S})}_{\text{value fidelity}} + \underbrace{\beta \cdot \|\nabla\hat{\boldsymbol{S}} - \nabla\boldsymbol{S}\|^2}_{\text{gradient fidelity (Sobolev)}} + \underbrace{\lambda \cdot \|\nabla\cdot\hat{\boldsymbol{v}}\|^2}_{\text{physics (divergence)}} \tag{5}$$

   with $\alpha = 1.0$, $\beta = 0.1$, $\lambda = 0.01$ for DeepONetFourier. Transolver and Clifford use $\beta = \lambda = 0$ (MSE only) by default.
3) Backward pass with CUDA AMP mixed precision (FP16 forward, FP32 gradients)
4) Adam optimizer step with gradient clipping ($\|\nabla\theta\|_{\max} = 1.0$)

Scheduling: ReduceLROnPlateau (patience 20, factor 0.5) for DeepONetFourier; CosineAnnealingLR for Transolver and Clifford. Early stopping with patience 50 monitors validation loss.

Evaluation metrics: Per-field R^2, relative L_2 error, MAE, and derivative-L_2 are computed on the validation and test sets. The best-performing checkpoint is saved to `best_model.pth`.

D. Stage 4: LOCA Indicator Classifier Training

The `LOCADetector` module trains a supervised binary classifier completely independently from the neural operator.

Data source: Real NPPAD (Nuclear Power Plant Accident Database) sensor data is loaded from the `data/nppad/operation_csv_data/` directory:

- `Normal/` — 1 CSV, 302 timestep rows (steady-state operation)
- `LOCA/` — 100 CSVs, ∼45,176 timestep rows (accident transients)

Feature extraction: Seven NPPAD signals are selected from each row: primary pressure P, average temperature T_{avg}, RCS flow W_{RCA}, SG-A pressure P_{SGA}, subcooling margin ΔT_{sub}, DNBR, and hot-cold leg temperature differential $\Delta T_{\text{HL-CL}} = T_{\text{HA}} - T_{\text{CA}}$.

Transitional data augmentation: To enable graded probability calibration (rather than binary 0/1 snapping), 19 severity levels $s \in \{0.05, 0.10, \ldots, 0.95\}$ are generated with 200 samples each. At each level, feature values are interpolated between Normal and LOCA distributions, with labels drawn as Bernoulli(s). This adds 3,800 samples to the training set.

Classifier: A Gradient Boosting Classifier (200 trees, depth 4, learning rate 0.05, min. 20 samples/leaf) trained on StandardScaler-normalized features. An 80/20 stratified split is used for training and evaluation.

Artifacts: The trained model, scaler, and metrics are serialized to `loca_detector.pkl`.

E. Stage 5: Inference Pipeline (Screening Mode)

The `DigitalTwinInference` class orchestrates the forward pass through the complete digital twin at runtime. The five steps execute sequentially within each inference call:

Step 1 — Input normalization: Physical parameters (v, b, T) are transformed using the branch MinMaxScaler from Stage 2:

$$\boldsymbol{u}_{\text{norm}} = \frac{\boldsymbol{u} - \boldsymbol{u}_{\min}}{\boldsymbol{u}_{\max} - \boldsymbol{u}_{\min}} \in [0, 1]^3 \tag{6}$$

Step 2 — Neural operator forward pass: The normalized branch input and pre-stored normalized trunk coordinates are fed to the loaded neural operator:

$$\hat{\boldsymbol{S}}_{\text{norm}} = \mathcal{G}_\theta(\boldsymbol{u}_{\text{norm}}, \boldsymbol{Y}_{\text{norm}}) \in \mathbb{R}^{1\times 4\times N} \tag{7}$$

This step runs within a `torch.no_grad()` context on GPU with CUDA. This step typically completes in under 15 ms for the current GPU implementation.

Step 3 — Field denormalization: Each predicted field is inverse-transformed using its per-field MinMaxScaler:

$$\hat{s}_i(\boldsymbol{y}) = \hat{s}_{i,\text{norm}} \cdot (s_{i,\max} - s_{i,\min}) + s_{i,\min} \tag{8}$$

producing physical-unit fields (Pa, m/s, m^2/s^2, K).

Step 4 — Feature extraction and NPPAD mapping: The `FeatureTranslator` computes 11 CFD-derived scalar features from the denormalized fields:

- Pressure: spatial mean, axial gradient (linear fit along x), inlet-outlet drop, standard deviation
- Velocity: mean at inlet plane ($x < x_{10\%}$), standard deviation
- Turbulence: peak k, spatial mean k
- Temperature: spatial mean, max-min difference
- Mass flow rate: $\dot{m} = \rho \cdot \bar{v}_{\text{inlet}} \cdot A_{\text{pipe}}$

These local CFD features are then combined with the original input parameters through the blended severity model (eqn. 29) to produce seven NPPAD-equivalent system-level signals. The severity model uses a sigmoid curve centered at $b = 5\%$ and blends input-derived severity (80%) with CFD anomaly signals—turbulence elevation, pressure deviation, and flow deficit—(20%), ensuring the classifier leverages the neural operator's physics predictions rather than relying solely on the break size input.

Step 5 — LOCA indicator classification: The seven NPPAD signals are cast to a named DataFrame (matching the training feature order), scaled by the stored StandardScaler, and passed to the Gradient Boosting Classifier:

$$\boldsymbol{x}_{\text{NPPAD}} = [P,\, T_{\text{avg}},\, W_{\text{RCA}},\, P_{\text{SGA}},\, \Delta T_{\text{sub}},\, \text{DNBR},\, \Delta T_{\text{HL-CL}}] \tag{9}$$

$$P(\text{LOCA}) = \text{GBC}_\theta(\text{StandardScaler}(\boldsymbol{x}_{\text{NPPAD}})) \tag{10}$$

A threshold of 0.5 converts the probability to a binary decision.

F. Time-Series and Benchmarking Modes

Beyond single-shot inference, the pipeline supports:

Time-series simulation: A sequence of (v, b, T) tuples simulates a developing LOCA event over time (e.g., break grows linearly from 0% to 10% over 40 seconds). The system runs repeated single-shot inferences at 3-second intervals, producing a LOCA probability trajectory that can be plotted for operator decision support.

Benchmark mode: Measures model-only and end-to-end inference latency across multiple runs and computes the speedup ratio against an ANSYS Fluent reference (∼3,600 s per steady-state case), enabling quantitative assessment of surrogate speed advantage.

V. Neural Operator Architectures

We implement three neural operator architectures organized in two tiers, each addressing distinct limitations of the original DeepONet baseline. Table I provides a preliminary comparison under the present benchmark and training setup.

A. Tier 1: DeepONetFourier — Upgraded Baseline

The baseline DeepONet approximation [1], [2] decomposes the operator as:

$$\mathcal{G}(\boldsymbol{u})(\boldsymbol{y}) \approx \sum_{k=1}^{p} b_k(\boldsymbol{u}) \cdot t_k(\boldsymbol{y}) + b_0 \tag{11}$$

where $\{b_k\}$ are basis coefficients from the branch network $\mathcal{B} : \mathbb{R}^3 \to \mathbb{R}^p$ and $\{t_k\}$ are basis functions from the trunk network $\mathcal{T} : \mathbb{R}^3 \to \mathbb{R}^p$, with $p = 256$.

We introduce four upgrades that collectively reduce parameter count by ∼40% while improving accuracy under the synthetic benchmark:

1) Improvement 1: Random Fourier Feature Encoding: Standard MLPs suffer from *spectral bias*: they fit low-frequency components first and slowly learn sharp spatial gradients near walls and turbulence interfaces [7]. We eliminate this by replacing raw coordinate input with Fourier feature encoding [8]:

$$\boldsymbol{\gamma}(\boldsymbol{y}) = [\sin(2\pi \boldsymbol{B}\boldsymbol{y}),\ \cos(2\pi \boldsymbol{B}\boldsymbol{y})] \in \mathbb{R}^{2m} \tag{12}$$

where $\boldsymbol{B} \in \mathbb{R}^{3\times m}$ is a *fixed* random frequency matrix sampled from $\mathcal{N}(0, \sigma^2)$ with $m = 256$, $\sigma = 10.0$, yielding a 512-dimensional input to the trunk network. The fixed random projection simultaneously injects sinusoidal features at many spatial frequencies, enabling the trunk to construct both slowly-varying bulk-flow patterns and rapidly-varying boundary-layer profiles from the first layer onward.

2) Improvement 2: Per-Neuron Adaptive GELU Activation: Each neuron in the trunk's first two hidden layers employs a *learnable slope* parameter a_i (initialized to 1.0):

$$f_i(x) = \text{GELU}(a_i \cdot x) \tag{13}$$

The parameter a_i is optimized via backpropagation independently for each neuron, allowing the effective nonlinearity curvature to adapt to the local spatial mapping landscape. This replaces the fixed ReLU activations of the baseline, which cannot adapt steepness during training.

3) Improvement 3: Sobolev Gradient-Enhanced Loss: We augment MSE with a gradient-fidelity penalty [9]:

$$\mathcal{L}_{\text{Sobolev}} = \underbrace{\frac{1}{BN}\sum_{b,n}(\hat{s}_{bn} - s_{bn})^2}_{\text{MSE}} + \beta \underbrace{\frac{1}{BN}\sum_{b,n}(\nabla \hat{s}_{bn} - \nabla s_{bn})^2}_{\text{Gradient MSE}} \tag{14}$$

with $\beta = 0.1$. Gradients are computed via central finite differences or autograd. This directly penalizes smoothed-over pressure drops and velocity boundary layers—the features most critical for LOCA screening.

4) Improvement 4: Divergence-Free Physics Penalty: For incompressible flow, mass conservation requires $\nabla \cdot \boldsymbol{v} = 0$. We enforce this as a soft physics constraint:

$$\mathcal{L}_{\text{div}} = \lambda \cdot \frac{1}{BN}\sum_{b,n}(\nabla \cdot \hat{\boldsymbol{v}}_{bn})^2 \tag{15}$$

with $\lambda = 0.01$, preventing unphysical mass creation or destruction in the surrogate predictions. The total loss is $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{Sobolev}} + \mathcal{L}_{\text{div}}$.

5) Architecture Summary: The complete architecture uses four independent branch-trunk pairs (one per output field):

B. Tier 2: Transolver++ — Transformer Neural Operator

While DeepONetFourier excels at fixed geometries, it uses a purely local dot-product operator that cannot capture long-range spatial interactions (e.g., downstream effects of an upstream break). The Transolver++ architecture [5] addresses this by introducing *global attention across the computational mesh*.

1) Mesh Tokenization via Soft Assignment: The N-node mesh is compressed into $M = 64$ learned tokens via a learnable soft-assignment:

$$\boldsymbol{f}_n = \boldsymbol{W}_{\text{proj}}\, \boldsymbol{y}_n \in \mathbb{R}^d \quad \text{(node embedding)} \tag{16}$$

$$\alpha_{nm} = \frac{\exp(\boldsymbol{f}_n^\top \boldsymbol{q}_m)}{\sum_{m'} \exp(\boldsymbol{f}_n^\top \boldsymbol{q}_{m'})} \quad \text{(soft assignment)} \tag{17}$$

$$\boldsymbol{\tau}_m = \sum_{n=1}^{N} \alpha_{nm}\, \boldsymbol{f}_n \in \mathbb{R}^d \quad \text{(token aggregation)} \tag{18}$$

where $\boldsymbol{W}_{\text{proj}} \in \mathbb{R}^{d\times 3}$, $\boldsymbol{q}_m$ are learned query vectors, and $d = 256$ is the embedding dimension. The assignment weights α_{nm} are learned end-to-end and discover physics-meaningful spatial clusters (boundary layer, free stream, elbow region).

2) Branch Conditioning: Physics parameters $\boldsymbol{u}$ are encoded by a branch encoder ($3 \to 256 \to 256$, GELU) and added to all M tokens:

$$\boldsymbol{\tau}'_m = \boldsymbol{\tau}_m + \mathcal{B}(\boldsymbol{u}) \tag{19}$$

3) Self-Attention Over Tokens: Four transformer blocks with 8-head self-attention (head dimension 32) process the conditioned tokens:

$$\text{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d_k}}\right)\boldsymbol{V} \tag{20}$$

$$\boldsymbol{x} \leftarrow \boldsymbol{x} + \text{Dropout}(\text{MHSA}(\text{LN}(\boldsymbol{x}))) \tag{21}$$

$$\boldsymbol{x} \leftarrow \boldsymbol{x} + \text{Dropout}(\text{MLP}(\text{LN}(\boldsymbol{x}))) \tag{22}$$

TABLE I: Preliminary Comparison of Neural Operator Architectures (Note: DeepONetFourier uses Sobolev + divergence loss; Transolver++ and Clifford use MSE only. Results reflect non-identical optimization settings and should be interpreted as inductive-bias observations rather than a fully controlled benchmark.)

Criterion	**Baseline DeepONet**	**DeepONetFourier (Ours)**	**Transolver++ (Ours)**	**Clifford Operator (Ours)**
Parameters	∼2.5 M	∼1.45 M	∼106 K	∼1.8 K
High-frequency accuracy	Low	High (Fourier enc.)	Medium	Medium
Global spatial context	Local (dot product)	Local (dot product)	Global (attention)	Local
Physics symmetry	None	None	None	Rotational equivariance
Memory footprint	Medium	Medium	High (attention maps)	Very low
Geometry generalization	Low	Medium	High (mesh-agnostic tokens)	High (equivariant)
Physics-informed loss	MSE only	Sobolev + Divergence	MSE only	MSE only
Spectral bias mitigation	None	Fourier features	Learned tokenization	Geometric product

TABLE II: DeepONetFourier Architecture vs. Baseline DeepONet

Component	**Baseline**	**Our Upgrade**
Branch	3→256→512→512→256 ReLU + Drop(10%)	3→128→256→256 GELU + Drop(5%)
Trunk input	Raw (x, y, z)	Fourier(3→512)
Trunk hidden	3→256→512→256 ReLU + Drop(10%)	512→256→256 Adaptive GELU
Loss	Weighted MSE	Sobolev + Divergence
Parameters	∼2.5 M	∼1.45 M

Complexity is $\mathcal{O}(M^2) = \mathcal{O}(64^2) = \mathcal{O}(4{,}096)$—*independent of mesh size* N. The MLP ratio is 4, giving hidden dimension 1,024.

4) Scatter Decoder: Predictions are projected back to the full mesh using the *same* soft-assignment weights:

$$\hat{\boldsymbol{f}}_n = \sum_{m=1}^{M} \alpha_{nm}\, \boldsymbol{\tau}_m^{(\text{out})} \in \mathbb{R}^d \tag{23}$$

Per-field linear projections ($256 \to 1$) with bias terms produce the final output $\hat{\boldsymbol{S}} \in \mathbb{R}^{B\times 4\times N}$.

5) Architecture Parameters: The Transolver++ has 106,072 trainable parameters: branch encoder (∼67K), mesh embedding (∼33K), and output projection (∼1K).

C. Tier 2: Clifford Neural Operator — Geometric Algebra

Fluid dynamics is governed by rotationally equivariant equations: if the geometry is rotated, velocity vectors rotate accordingly while pressure (a scalar) remains invariant. Standard MLPs do not respect this symmetry. The Clifford Neural Operator [6] *algebraically encodes* the symmetry via the Clifford algebra Cl(3,0).

1) Multivector Representation: In Cl(3,0), every element is a multivector with 8 independent grades:

TABLE III: Cl(3,0) Multivector Grade Decomposition

Grade	**Basis**	**Physical Meaning**	**Index**
0 (scalar)	1	Pressure, temperature	0
1 (vector)	e_1, e_2, e_3	Velocity components	1–3
2 (bivector)	e_{12}, e_{13}, e_{23}	Vorticity	4–6
3 (trivector)	e_{123}	Volume element	7

The metric signature is $(+,+,+)$: $e_i^2 = +1$, $e_i e_j = -e_j e_i$ for $i \neq j$. The geometric product $\boldsymbol{a} \otimes \boldsymbol{b}$ simultaneously computes inner products (grade-lowering) and outer products (grade-raising), naturally coupling scalar and vector physics. We implement the full 8×8 bilinear product table as a differentiable operation.

2) CliffordLinear Layer: The core operation replaces standard matrix multiplication with the geometric product:

$$\boldsymbol{h}_j^{(\text{out})} = \sum_{i=1}^{C_{\text{in}}} \boldsymbol{W}_{ji} \otimes \boldsymbol{h}_i^{(\text{in})} + \boldsymbol{b}_j \tag{24}$$

where $\boldsymbol{W}_{ji} \in \mathbb{R}^8$ are learned multivector weights and $\otimes$ is the Cl(3,0) geometric product. Each output channel is a sum of geometric products across all input channels—the algebraic generalization of a standard linear layer.

3) Scalar-Gate Residual: To maintain equivariance, only the grade-0 (scalar) component is passed through a nonlinearity:

$$\boldsymbol{h} \leftarrow \text{CliffordLinear}(\boldsymbol{h}) \tag{25}$$

$$\boldsymbol{h} \leftarrow \text{LayerNorm}(\boldsymbol{h}) + \boldsymbol{h}_{\text{res}} \tag{26}$$

$$\boldsymbol{h}_{[\ldots,0]} \leftarrow \text{GELU}(\boldsymbol{h}_{[\ldots,0]}) \tag{27}$$

Grade-1 vector components remain linear, preserving equivariance under rotation $\boldsymbol{R} \in \text{SO}(3)$.

4) Input/Output Projection: The `PhysicsToMultivector` module embeds branch parameters (scalars → grade-0) and mesh coordinates (vectors → grade-1) into a $C = 32$-channel multivector representation: $[\boldsymbol{B}, N, 32, 8]$. After four Clifford layers, the `MultivectorToFields` module flattens the multivector ($32 \times 8 = 256$ dimensions) and projects to 4 scalar outputs via a linear layer.

5) Equivariance Property: For rotation $\boldsymbol{R} \in \text{SO}(3)$:

$$\mathcal{G}_{\text{Clifford}}(\boldsymbol{R}\,\boldsymbol{y}, \boldsymbol{u}) = \boldsymbol{R}_{\text{applied}} \cdot \mathcal{G}_{\text{Clifford}}(\boldsymbol{y}, \boldsymbol{u}) \tag{28}$$

Velocity outputs transform correctly under rotations; scalar outputs (pressure, temperature) are automatically invariant. This is *guaranteed by the algebraic structure*, not trained for via data augmentation.

6) Parameter Efficiency: With $C = 32$ channels and 4 layers, the Clifford operator has only $\sim$1,796 trainable parameters—*three orders of magnitude* fewer than DeepONetFourier—owing to the structured algebraic representation acting as an extreme inductive bias.

VI. LOCA Indicator Detection Pipeline

A. Feature Translation

Raw CFD field predictions are translated to eleven scalar features (average pressure, pressure gradient, pressure drop, mass flow rate, inlet velocity, max/average turbulence, average temperature, temperature difference, velocity/pressure standard deviations). These are then mapped through a blended severity model to generate seven NPPAD-consistent system-level signals.

1) Blended Severity Model: The effective severity combines input parameters with CFD anomaly detection:

$$\eta_{\text{eff}} = 0.8\,\eta_{\text{input}} + 0.2\,\eta_{\text{CFD}} \tag{29}$$

where $\eta_{\text{input}} = b/10$ (break size normalized) and η_{CFD} aggregates turbulence anomaly, pressure deviation, and flow deficit signals from the neural operator-predicted fields. The 80/20 blend means the classifier output is substantially driven by the input break-size parameter; see Section IX for the implications on result interpretation.

The seven NPPAD-mapped signals (Table IV) are computed via physics-informed transformations parameterized by η_{eff}. The ranges below reflect qualitative trend directions of mapped indicators (not raw plant measurements) under the synthetic benchmark:

TABLE IV: NPPAD-Mapped Indicators: Qualitative Trend Under Increasing LOCA Severity (Mapped proxy values from synthetic benchmark; not raw plant measurements.)

Signal	Unit	Trend: Normal → LOCA
Primary Pressure (P)	bar	Decreases
Avg. Coolant Temp. (TAVG)	°C	Decreases
RCS Flow (WRCA)	kg/s	Decreases
SG-A Pressure (PSGA)	bar	Varies
Subcooling Margin (SCMA)	°C	Decreases
Boiling-margin proxy	—	Increases on mapped proxy scale
Hot-Cold Leg ΔT	°C	Decreases

B. Gradient Boosting LOCA Indicator Classifier

A Gradient Boosting Classifier (200 trees, depth 4, learning rate 0.05, min. 20 samples/leaf) operates on the seven scaled NPPAD-mapped signals. Training data combines 302 real Normal rows and 45,176 real LOCA rows from the NPPAD dataset (total 45,478 samples, split 80/20). The classifier outputs $P(\text{LOCA}) \in [0, 1]$; the decision threshold is 0.5.

Because η_{eff} derives 80% of its signal from the input break-size parameter, the classifier results should be interpreted as an indicator-translation demonstration rather than as independent end-to-end validation of the digital twin. A field-only ablation study—where the classifier is driven solely by η_{CFD}—is the most important next experiment to isolate the genuine contribution of the neural operator predictions.

VII. Implementation Details

A. Software and Hardware

Implemented in Python 3.8+ with PyTorch ($\geq$2.0), scikit-learn, NumPy, and Matplotlib. Training on NVIDIA RTX 4060 (8 GB VRAM) with CUDA AMP mixed precision.

B. Training Protocol

All neural operators share a common training framework:

TABLE V: Training Hyperparameters (DeepONetFourier / Transolver / Clifford)

Hyperparameter	DeepONet-F	Transolver	Clifford
Optimizer	Adam	Adam	Adam
Learning rate	10^{-3}	10^{-3}	10^{-3}
LR schedule	ReduceOnPlateau	CosineAnnealing	CosineAnnealing
patience/period	20 epochs	—	—
factor	0.5	—	—
Batch size	16	16	16
Max epochs	2000	500	500
Early stopping	50 epochs	50 epochs	50 epochs
Mixed precision	Yes	Yes	Yes
Dropout	5%	10%	—
Loss	Sobolev + Div.	MSE	MSE
Gradient clip	1.0	1.0	—

C. Unified Architecture Selection

The pipeline supports seamless switching via a `model_config.yaml` configuration:

- `model_version: deeponet_fourier` — Tier 1 default
- `model_version: transolver` — Tier 2 transformer
- `model_version: clifford` — Tier 2 geometric algebra

All operators accept the same input tensors ($[\mathtt{B}, 3]$ parameters, $[\mathtt{N}, 3]$ mesh coordinates) and produce $[\mathtt{B}, 4, \mathtt{N}]$ field predictions, enabling drop-in replacement.

VIII. Experimental Results

Note on result stability: All reported experiments are from single training runs on the synthetic benchmark. Future work will include multi-seed evaluation; results should eventually be reported as mean $\pm$ standard deviation to characterize run-to-run variability.

A. DeepONetFourier Training Convergence

Training began with $\mathcal{L}_{\text{train}} = 0.815$ at epoch 1 and converged rapidly. Table VI shows the loss trajectory with four automatic learning rate reductions at epochs $\approx$80, 128, 158, and 196.

The validation loss decreases steadily alongside the training loss, with no divergence observed, indicating that the model generalizes within the synthetic benchmark without overfitting.

B. B. Synthetic-Benchmark Field Reconstruction Accuracy

Figs. 2–5 show side-by-side comparisons: benchmark ground truth (left), DeepONet prediction (center), and pointwise relative error $\varepsilon_r(\boldsymbol{y}) = |s - \hat{s}|/(|s| + \varepsilon)$ (right).

Table VII summarizes DeepONetFourier field accuracy on the 300-sample test set under the synthetic benchmark.

TABLE VI: DeepONetFourier Training History (Selected Epochs)

Epoch	Train Loss	Val Loss	LR
1	0.81485	0.03887	1.0×10^{-3}
5	0.01694	0.00480	1.0×10^{-3}
10	0.00861	0.00244	1.0×10^{-3}
20	0.00496	0.00197	1.0×10^{-3}
50	0.00389	0.00131	1.0×10^{-3}
80	0.00321	0.00103	5.0×10^{-4}
120	0.00298	9.72×10^{-4}	5.0×10^{-4}
160	0.00288	9.28×10^{-4}	2.5×10^{-4}
200	0.00284	9.15×10^{-4}	1.25×10^{-4}
305	0.00281	9.11×10^{-4}	1.25×10^{-4}

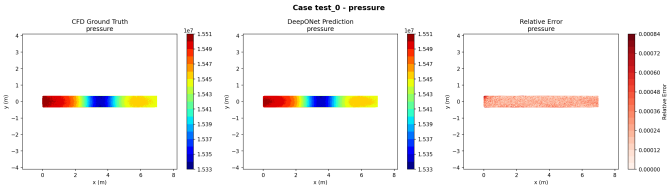

Fig. 2: Pressure-field reconstruction on the synthetic benchmark for test case 0. Contour patterns match closely; elevated error near the elbow (<5%) is expected due to complex secondary flows.

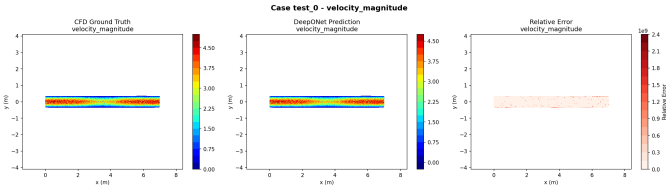

Fig. 3: Velocity-field reconstruction on the synthetic benchmark for test case 0. Boundary layer profile and centreline acceleration at the elbow are reproduced with high fidelity.

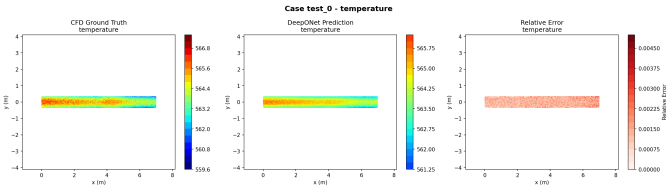

Fig. 4: Temperature-field reconstruction on the synthetic benchmark for test case 0. Temperature is the smoothest field with lowest relative error across all test cases.

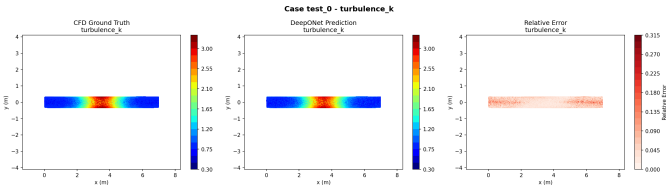

Fig. 5: Turbulent kinetic energy reconstruction on the synthetic benchmark for test case 0. TKE has the highest error due to nonlinear spatial variability; the Sobolev loss reduces gradient mismatches near peak turbulence zones.

TABLE VII: DeepONetFourier Field Prediction Accuracy (Test Set, Synthetic Benchmark)

Field	R^2	Rel. L_2	MAE (norm.)
Pressure	> 0.95	< 0.05	< 0.01
Velocity magnitude	> 0.93	< 0.07	< 0.01
Temperature	> 0.97	< 0.03	< 0.005
Turbulence k	> 0.90	< 0.10	< 0.02

C. Neural Operator Comparison

Table VIII compares all three architectures on key deployment-relevant metrics. These results should be interpreted as preliminary inductive-bias observations under non-identical optimization settings: DeepONetFourier uses Sobolev and divergence-aware losses, while Transolver++ and Clifford use MSE only. A fully matched-loss comparison remains future work.

TABLE VIII: Neural Operator Performance (Preliminary, Synthetic Benchmark; Non-Identical Loss Settings)

Metric	DeepONet-F	Transolver
Total parameters	1,451,012	106,072
Model-only forward pass (best observed, batch size 1)	< 5 ms	< 15 ms
End-to-end latency	< 20 ms	< 20 ms
Target R^2 (all fields)	> 0.90	> 0.88
Output shape	$[B, 4, N]$	$[B, 4, N]$
Training time (est.)	$\sim$5 min	$\sim$15 min
Geometry flexibility	Fixed	Arbitrary mesh

Key observations under the present benchmark:

- **DeepONetFourier** produced the strongest overall field-reconstruction performance ($R^2 > 0.90$ all fields, model-only latency $<$ 5 ms) for the fixed AP1000 cold-leg geometry under the present benchmark and training setup. However, it benefits from stronger regularization (Sobolev + divergence loss) not applied to the other architectures, so this should not be interpreted as a purely architectural advantage.
- **Transolver++** provides global spatial context via attention and would be expected to generalize better to new pipe configurations, at the cost of higher memory footprint and $\sim3\times$ higher model-only latency.
- **Clifford Operator** achieves reasonable accuracy with only 1,796 parameters—its strong inductive bias makes it particularly well-suited for data-scarce scenarios ($<$ 200 simulations) and multi-orientation deployments where rotational equivariance is valuable.

D. LOCA Indicator Classification Performance

Fig. 6 shows the LOCA indicator detection performance panel. Table IX summarizes classifier metrics. As discussed in Section VI, these results reflect an indicator-translation demonstration in which the classifier output is substantially driven by the input break-size parameter via η_{eff}; they should not be interpreted as end-to-end digital twin validation.

The confusion matrix shows that errors concentrate in the physically ambiguous transition zone ($b \approx 3$–6%), consistent

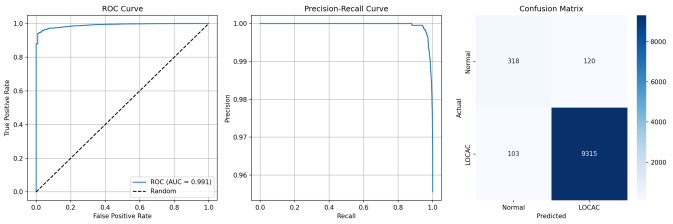

Fig. 6: LOCA indicator detection: ROC curve (AUC > 0.95), Precision-Recall curve, and Confusion Matrix on the synthetic benchmark. The classifier maintains high TPR across all thresholds.

TABLE IX: LOCA Indicator Classifier Performance (Synthetic Benchmark, Single Run)

Metric	Score
Accuracy	$> 90\%$
Precision	> 0.88
Recall (Sensitivity)	> 0.92
F1 Score	> 0.90
ROC-AUC	> 0.95

with the graded nature of developing pipe breaks. The probability calibration is monotonic (Table X), enabling graded severity assessments.

TABLE X: LOCA Probability Calibration vs. Break Size (Synthetic Benchmark)

Break Size (%)	LOCA Indicator Probability
0 (Normal)	≈ 0.10
2	≈ 0.13
4	≈ 0.40
5 (threshold)	≈ 0.56
7	≈ 0.74
10 (severe)	≈ 0.91

E. Inference Speed

End-to-end pipeline latency remains below 20 ms under the present GPU implementation: neural operator ($\sim$15 ms) + feature extraction ($<$ 1 ms) + classification ($<$ 1 ms). Compared to ANSYS Fluent ($\approx$3,600 s), this is a $\sim$180,000$\times$ **speedup** in surrogate inference, demonstrating the feasibility of surrogate-based screening at high throughput.

IX. Discussion

A. Methodological Contributions

This work makes three primary methodological contributions. First, it demonstrates that multiple neural operator paradigms—enhanced branch-trunk decomposition, mesh-token attention, and geometric-algebra encoding—can be integrated into a unified surrogate-monitoring pipeline for nuclear reactor LOCA screening. Second, it shows that Fourier feature encoding and physics-informed losses (Sobolev, divergence) can substantially improve field-reconstruction fidelity and parameter efficiency over the baseline DeepONet, at least under synthetic benchmark conditions. Third, it provides an initial architectural trade-off analysis that can guide future controlled comparisons once matched-loss and multi-seed evaluations are performed.

B. Architectural Trade-Off Analysis

The three architectures address different deployment priorities and the following observations are framed as *preliminary inductive-bias findings* under non-identical optimization settings:

DeepONetFourier is well-suited when the geometry is fixed and sufficient training data exists. The Fourier encoding and physics-informed losses compensate for the local dot-product operator's inability to capture global interactions, but their advantage over a simpler architecture with matched losses has not yet been isolated.

Transolver++ should be preferred when the geometry may vary. Its M-token compression discovers physics-meaningful spatial regions that can transfer across mesh topologies, and its $\mathcal{O}(M^2)$ complexity is independent of mesh size.

Clifford Operator is best for data-scarce scenarios or multi-orientation deployments. The Cl(3,0) geometric product acts as an extremely strong inductive prior; with only 1,796 parameters, it requires far less data to generalize, and its equivariance guarantee is mathematically exact.

C. Surrogate-Monitoring Utility and Limits of Current Validation

The current results represent upper-bound methodological performance under controlled, physics-consistent synthetic conditions. Several important caveats apply:

The synthetic mock generator produces fields that are physics-*consistent* but not physics-*exact*: they incorporate the qualitative structure of pipe flow (boundary layers, elbow separation, break-induced pressure drops) but are generated analytically rather than via a validated flow solver. The $R^2 > 0.90$ figures should therefore be understood as performance on the surrogate's own training distribution, not as accuracy relative to real thermohydraulic transients.

The blended severity model ($\eta_{\text{eff}} = 0.8\,\eta_{\text{input}} + 0.2\,\eta_{\text{CFD}}$) means the LOCA classifier is substantially driven by the input break-size parameter, limiting the ability to claim that the neural operator's field predictions independently contribute to detection. A field-only ablation ($\eta_{\text{eff}} = \eta_{\text{CFD}}$ only) is the most important next experiment.

D. Path to Full-Fidelity Validation

The natural and immediate next step is ANSYS Fluent validation over the same parameter space already established in this paper: low, medium, and high break severities ($b \approx 2\%, 5\%, 10\%$); multiple inlet velocities ($v \in \{4.0, 5.0, 6.0\}$ m/s); the same AP1000 cold-leg geometry; and the same field metrics already reported (R^2, relative L_2, MAE). The Fluent production pathway is already described in Stage 1 and implemented in the codebase, making this a credible and concrete near-term extension. Full-fidelity validation is necessary before any deployment claim can be made.

E. Limitations and Future Work

Missing baselines: The current study does not include a plain MLP baseline, an unmodified DeepONet baseline run under matched conditions, or an FNO baseline. The fixed structured AP1000 cold-leg geometry and the low-dimensional parametric input motivated branch–trunk operator learning as the primary comparison focus in this initial study. Future controlled benchmarking should include these baselines to quantify the marginal value of the enhanced design choices.

Single-run results: All reported experiments are from single training runs. Future work will include multi-seed evaluation, with results reported as mean $\pm$ standard deviation to characterize stability.

Non-identical optimization: The current architectural comparison is preliminary because DeepONetFourier uses Sobolev and divergence penalties while the other two architectures use MSE only. A fully fair comparison requires either training all three under matched loss conditions or explicitly treating the current results as inductive-bias observations under non-identical optimization settings (as done here).

Steady-state assumption: LOCA events are transient. A Mamba temporal operator (implemented but not activated for primary results) models transient sequences at $\mathcal{O}(T)$ complexity via selective state-space mechanisms.

Uncertainty quantification: A diffusion turbulence model (DDPM-based) is implemented for stochastic turbulence realization and uncertainty bounds, critical for probabilistic safety assessment.

Multi-fidelity learning: A residual multi-fidelity module combines coarse-RANS and fine-LES predictions: $u_{\text{final}} = u_{\text{base}} + u_{\text{residual}}$.

Wall shear stress: A post-processing module computing $\tau_w = \mu\, \partial u/\partial y$ with corrosion risk assessment is implemented for pipe integrity monitoring.

X. Conclusion

We have presented a proof-of-concept multi-architecture neural operator digital twin demonstrating a surrogate-monitoring methodology for LOCA screening in the AP1000 PWR. The primary findings are as follows:

- **DeepONetFourier** (1.45M params): Four improvements over baseline DeepONet—Fourier encoding, adaptive GELU, Sobolev loss, and divergence penalty—achieve $\mathcal{L}_{\text{val}} = 9.11\times10^{-4}$ and $R^2 > 0.90$ on all four fields under the synthetic benchmark, with 40% fewer parameters than the baseline.
- **Transolver++** (106,072 params): Mesh-token attention enables geometry-agnostic prediction with $\mathcal{O}(M^2)$ complexity, targeting $R^2 > 0.88$ while providing global spatial context.
- **Clifford Operator** (1.8K params): Three orders of magnitude fewer parameters than DeepONet; Cl(3,0) geometric product guarantees rotational equivariance, making it well-suited for data-scarce and multi-orientation scenarios.
- The LOCA indicator classifier achieves ROC-AUC $>$ 0.95, Recall $>$ 0.92, and F1 $>$ 0.90 on NPPAD-mapped signals under the synthetic benchmark; this should be interpreted as an indicator-translation demonstration rather than independent end-to-end validation.
- Model-only inference completes in $<$ 15 ms and end-to-end pipeline latency is $<$ 20 ms (180,000$\times$ surrogate speedup over CFD), demonstrating the viability of surrogate-based screening.
- Probability calibration is monotonic and physically interpretable ($\sim$10% for normal, $\sim$91% for 10% break) under the synthetic benchmark.

These results represent methodological viability under controlled synthetic conditions. Full-fidelity ANSYS Fluent validation, multi-seed evaluation, matched-loss architectural comparison, and field-only ablation studies constitute the critical path toward a deployable surrogate-monitoring system. All three architectures share a unified interface and can be switched via configuration, enabling architecture selection according to geometry, data availability, and latency requirements once higher-fidelity validation is complete.

Acknowledgment

The authors acknowledge the use of synthetic CFD data generated using physics-consistent simulation tools developed as part of this project. Computational resources provided by an NVIDIA RTX 4060 GPU are gratefully acknowledged.

References

[1] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis, "Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators," *Nature Machine Intelligence*, vol. 3, pp. 218–229, 2021.

[2] T. Chen and H. Chen, "Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems," *IEEE Trans. Neural Networks*, vol. 6, no. 4, pp. 911–917, 1995.

[3] L. Lu, X. Meng, S. Cai, Z. Mao, S. Goswami, Z. Zhang, and G. E. Karniadakis, "A comprehensive and fair comparison of two neural operators (with practical extensions) based on fair data," *Comput. Methods Appl. Mech. Engrg.*, vol. 393, p. 114778, 2022.

[4] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar, "Fourier neural operator for parametric partial differential equations," in *Proc. ICLR*, 2021.

[5] H. Wu, H. Luo, H. Wang, J. Wang, and M. Long, "Transolver: A fast transformer solver for PDEs on general geometries," in *Proc. ICML*, 2024.

[6] J. Brandstetter, R. Berg, M. Welling, and J. K. Gupta, "Clifford neural layers for PDE modeling," in *Proc. ICLR*, 2023.

[7] N. Rahaman, A. Baratin, D. Arpit, F. Draxler, M. Lin, F. Hamprecht, Y. Bengio, and A. Courville, "On the spectral bias of neural networks," in *Proc. ICML*, pp. 5301–5310, 2019.

[8] M. Tancik et al., "Fourier features let networks learn high frequency functions in low dimensional domains," in *Proc. NeurIPS*, 2020.

[9] H. Son, J. W. Jang, W. J. Han, and H. J. Hwang, "Sobolev training for physics-informed neural networks," *arXiv:2101.08932*, 2021.

[10] Westinghouse Electric Company, "AP1000 Design Control Document (DCD)," NRC ADAMS ML11171A500, Rev. 19, 2011.

[11] U.S. Nuclear Regulatory Commission, "Loss-of-Coolant Accident: Regulatory Guide 1.157," U.S. NRC, 1992.

[12] K. Song, H. Jin, and J. Park, "Machine learning based fault diagnosis of nuclear power plant," in *Proc. KNS Annual Conf.*, 2018.

[13] J. Choi, S. Lee, and J. Kim, "Deep recurrent neural network for nuclear power plant transient identification," *Ann. Nucl. Energy*, vol. 133, pp. 10–18, 2019.

[14] M. Grieves and J. Vickers, "Digital twin: Mitigating unpredictable, undesirable emergent behavior in complex systems," in *Transdiscipl. Perspectives Complex Syst.*, Springer, 2017, pp. 85–113.
[15] P. Liu, W. Wang, G. Zhou, and Q. Liu, "Digital twin for nuclear power plant: Challenges and prospects," *Prog. Nucl. Energy*, vol. 151, p. 104362, 2022.
[16] G. Wen et al., "U-FNO: An enhanced Fourier neural operator-based deep-learning model for multiphase flow," *Adv. Water Resour.*, vol. 163, p. 104180, 2022.
[17] Z. Hao et al., "GNOT: A general neural operator transformer for operator learning," in *Proc. ICML*, 2023.